

Web Technology (310252) — Viva Questions & Answers

SPPU | Semester VI | Computer Engineering

Practical 1: Website Evaluation Case Study

Q1. What is website evaluation?

Ans: Website evaluation is the systematic process of assessing a website's quality, usability, performance, design, and content to determine how well it meets its intended goals and user needs.

Q2. Why is website evaluation important before development?

Ans: It helps identify best practices, avoid common mistakes, understand target audience expectations, benchmark against competitors, and define clear design and functionality goals before investing in development.

Q3. What are the key parameters for evaluating a website?

Ans: Key parameters include: usability, UI/UX design, loading speed, mobile responsiveness, cross-browser compatibility, accessibility, SEO, content quality, navigation structure, and security.

Q4. What is usability in web design?

Ans: Usability refers to how easily and efficiently users can interact with a website to achieve their goals. It covers learnability, efficiency, memorability, error prevention, and user satisfaction (ISO 9241 standard).

Q5. What is user experience (UX)?

Ans: UX (User Experience) is the overall feeling and satisfaction a user has while interacting with a website. It encompasses usability, accessibility, performance, design, and content relevance.

Q6. What is user interface (UI)?

Ans: UI (User Interface) refers to the visual and interactive elements of a website — buttons, menus, forms, layouts, typography, colors — through which users interact with the system.

Q7. Difference between UI and UX?

Ans: UI is the look and feel (visual design, layout, interactive elements), while UX is the overall experience (ease of use, satisfaction, efficiency). UI is a subset of UX. Good UI contributes to good UX, but UX involves research, testing, and strategy beyond visual design.

Q8. What is responsive design?

Ans: Responsive design is a web design approach that makes a website automatically adapt its layout, images, and content to fit different screen sizes (desktop, tablet, mobile) using fluid grids, flexible images, and CSS media queries.

Q9. What is accessibility in websites?

Ans: Web accessibility means designing websites usable by people with disabilities (visual, auditory, motor, cognitive). It follows WCAG (Web Content Accessibility Guidelines), including alt text, keyboard navigation, color contrast, and screen reader support.

Q10. What is navigation structure?

Ans: Navigation structure is the organization and hierarchy of links and menus that guide users through a website. Good navigation should be intuitive, consistent, and allow users to find information with minimal clicks.

Q11. Why is loading speed important?

Ans: Loading speed directly affects user experience, bounce rate, and SEO rankings. Google recommends pages load within 2–3 seconds. Slow pages increase abandonment rates and negatively impact conversions and search ranking.

Q12. What is bounce rate?

Ans: Bounce rate is the percentage of visitors who leave a website after viewing only one page without interacting further. A high bounce rate may indicate poor UX, irrelevant content, slow load time, or poor design.

Q13. What is SEO-friendly design?

Ans: SEO-friendly design incorporates practices that help search engines index and rank a website effectively: proper use of heading tags, semantic HTML, alt attributes for images, fast loading, mobile responsiveness, clean URLs, and sitemap.xml.

Q14. What makes a website visually appealing?

Ans: Visual appeal depends on consistent color scheme, good typography, appropriate use of whitespace, high-quality images, balanced layout, clear visual hierarchy, and alignment with brand identity.

Q15. What are common mistakes in website design?

Ans: Common mistakes include: cluttered layouts, inconsistent design, poor color contrast, lack of mobile optimization, slow loading, broken links, missing CTAs, hard-to-read fonts, and absence of error handling.

Q16. How do colors affect user perception?

Ans: Colors evoke emotions and influence decisions. For example, blue conveys trust, red urgency, green health/success. Color theory helps in choosing palettes that align with brand values and guide user behavior effectively.

Q17. What is consistency in web design?

Ans: Consistency means using the same fonts, colors, button styles, spacing, and navigation patterns throughout the website. It reduces cognitive load, builds trust, and makes the site predictable and professional.

Q18. What is content hierarchy?

Ans: Content hierarchy refers to the arrangement of information based on importance, using visual cues like size, color, weight, and positioning to guide users' attention from most to least important content.

Q19. What is call-to-action (CTA)?

Ans: A CTA is a design element (button, link, or banner) that prompts users to take a specific action, such as 'Sign Up', 'Buy Now', or 'Contact Us'. Effective CTAs are prominent, clear, and action-oriented.

Q20. What is mobile optimization?

Ans: Mobile optimization ensures a website performs well on smartphones and tablets: fast load times, touch-friendly elements, readable fonts, proper scaling, and simplified navigation for small screens.

Q21. What is cross-browser compatibility?

Ans: Cross-browser compatibility ensures a website looks and functions correctly across different browsers (Chrome, Firefox, Safari, Edge) and their various versions, despite differences in rendering engines.

Q22. What is heuristic evaluation?

Ans: Heuristic evaluation is a usability inspection method where evaluators assess a UI against established usability principles (heuristics), such as Nielsen's 10 Usability Heuristics, to identify usability issues.

Q23. How do you compare two websites objectively?

Ans: Use a structured rubric with criteria like usability, speed (PageSpeed Insights), SEO (Lighthouse), design consistency, accessibility (WAVE tool), content quality, and user feedback to score and compare both sites.

Q24. What is cognitive load?

Ans: Cognitive load is the mental effort required to use a website. Good design minimizes cognitive load through simple language, clear navigation, visual chunking, and predictable interactions so users can focus on their tasks.

Q25. How can poor design affect business outcomes?

Ans: Poor design leads to high bounce rates, low conversions, damaged brand reputation, poor SEO ranking, customer distrust, and ultimately revenue loss. Good UX can increase conversion rates significantly.

Q26. What tools can be used for website evaluation?

Ans: Common tools include: Google Lighthouse (performance/SEO/accessibility), GTmetrix (speed), WAVE (accessibility), SEMrush (SEO), Hotjar (user behavior), BrowserStack (compatibility), and Google Analytics (traffic).

Q27. What is user feedback analysis?

Ans: User feedback analysis involves collecting and studying user opinions (surveys, reviews, heatmaps, session recordings, A/B tests) to identify pain points and areas of improvement in the website.

Q28. How would you redesign a bad website?

Ans: Steps: (1) Audit the existing site, (2) Define target audience and goals, (3) Create wireframes, (4) Apply responsive design principles, (5) Optimize performance and SEO, (6) Test with real users, (7) Iterate based on feedback.

Q29. What are modern web design trends?

Ans: Current trends include: dark mode, glassmorphism, neumorphism, micro-animations, AI-powered personalization, PWA (Progressive Web Apps), voice UI, minimalistic design, and accessibility-first development.

Q30. What conclusions can be drawn from evaluating multiple websites?

Ans: Evaluation reveals industry best practices, user expectations for specific domains, common UX patterns, performance benchmarks, and areas where competitors are lacking — all guiding better design decisions.

Practical 2: HTML + CSS Webpage (index.htm)

Q1. What is HTML?

Ans: HTML (HyperText Markup Language) is the standard markup language used to create and structure web pages. It defines the skeleton of a webpage using elements represented by tags like `<html>`, `<head>`, `<body>`, `<p>`, etc.

Q2. What is the structure of an HTML document?

Ans: A basic HTML document has: `<!DOCTYPE html>` declaration, `<html>` root element containing `<head>` (metadata: title, CSS links, meta tags) and `<body>` (visible content: headings, paragraphs, images, forms).

Q3. What are tags?

Ans: Tags are markup elements enclosed in angle brackets that define the structure and content of HTML documents. They come in pairs: opening tag `<tagname>` and closing tag `</tagname>`, or as self-closing tags like `<br/>`, `<img/>`.

Q4. What are attributes?

Ans: Attributes provide additional information about HTML elements. They are written inside the opening tag as name-value pairs. Example: `<img src='image.jpg' alt='Description'>` — here `src` and `alt` are attributes.

Q5. What are heading tags?

Ans: Heading tags `<h1>` to `<h6>` define headings with decreasing importance/size. `<h1>` is the main heading (largest), `<h6>` is the smallest. They create content hierarchy and are important for SEO and accessibility.

Q6. What is the purpose of `<p>` tag?

Ans: The `<p>` tag defines a paragraph of text. Browsers automatically add space (margin) before and after each paragraph. It is a block-level element and is one of the most commonly used HTML tags.

Q7. What is the `<a>` tag?

Ans: The `<a>` (anchor) tag creates hyperlinks. The `href` attribute specifies the URL. Example: `<a href='https://example.com'>Click</a>`. The `target='_blank'` attribute opens the link in a new tab.

Q8. What is an image tag?

Ans: The `<img>` tag embeds images in an HTML page. It is a self-closing tag. Key attributes include `src` (image path), `alt` (alternative text for accessibility/SEO), `width`, and `height`. Example: `<img src='photo.jpg' alt='Photo'>`.

Q9. What are lists in HTML?

Ans: HTML provides three types of lists: Ordered List (`<ol>`) — numbered items; Unordered List (`<ul>`) — bulleted items; Definition List (`<dl>`) — terms and descriptions. List items are defined using the `<li>` tag.

Q10. Difference between ordered and unordered lists?

Ans: An ordered list (`<ol>`) displays items with sequential numbers or letters (1, 2, 3 or a, b, c). An unordered list (`<ul>`) displays items with bullet points. Both use `<li>` for each list item.

Q11. What is a table in HTML?

Ans: HTML tables organize data in rows and columns using `<table>`, `<tr>` (row), `<th>` (header cell), and `<td>` (data cell) tags. Attributes like `border`, `colspan`, and `rowspan` control appearance and cell spanning.

Q12. What are forms in HTML?

Ans: HTML forms collect user input using the `<form>` element containing input elements like `<input>`, `<textarea>`, `<select>`, `<button>`. The `action` attribute specifies where to send data; `method` attribute specifies GET or POST.

Q13. What is CSS?

Ans: CSS (Cascading Style Sheets) is a stylesheet language used to control the visual presentation of HTML elements — including layout, colors, fonts, spacing, borders, and animations — separating design from content.

Q14. Types of CSS?

Ans: There are three types: (1) Inline CSS — applied directly to an element using the `style` attribute; (2) Internal CSS — written within `<style>` tags in the `<head>` section; (3) External CSS — written in a separate `.css` file linked via `<link>`.

Q15. Difference between inline, internal, and external CSS?

Ans: Inline CSS has highest specificity and is hardest to maintain. Internal CSS applies styles to one page only. External CSS is best practice — it applies to multiple pages, improves maintainability, and separates content from design.

Q16. What is a class selector?

Ans: A class selector targets HTML elements with a specific class attribute value. It is written with a dot prefix: `.classname { }`. Multiple elements can share the same class, making it reusable across the page.

Q17. What is an ID selector?

Ans: An ID selector targets a unique HTML element with a specific id attribute. It is written with a hash prefix: `#idname { }`. IDs must be unique within a page and have higher specificity than class selectors.

Q18. What is the CSS box model?

Ans: Every HTML element is treated as a rectangular box. The CSS box model consists of: Content (actual text/image), Padding (space between content and border), Border (surrounds padding), and Margin (space outside border).

Q19. What is margin and padding?

Ans: Padding is the space between an element's content and its border (inside the element). Margin is the space outside the element's border, separating it from adjacent elements. Both can be set individually for each side (top, right, bottom, left).

Q20. What is display property?

Ans: The display property controls how an element is rendered: block (takes full width, starts on new line — `<div>`, `<p>`), inline (takes only needed width — `<span>`, `<a>`), inline-block, flex, grid, and none (hides element).

Q21. What is position property?

Ans: The position property specifies how an element is positioned: static (default), relative (relative to its normal position), absolute (relative to nearest positioned ancestor), fixed (relative to viewport), and sticky (hybrid).

Q22. What are semantic elements?

Ans: Semantic HTML elements clearly describe their meaning: `<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`, `<figure>`. They improve accessibility, SEO, and code readability compared to generic `<div>` elements.

Q23. What is responsive design?

Ans: Responsive design creates webpages that adapt to different screen sizes using fluid grids, flexible images, and CSS media queries. The meta viewport tag is essential: `<meta name='viewport' content='width=device-width, initial-scale=1'>`.

Q24. What are media queries?

Ans: Media queries apply different CSS styles based on device characteristics like screen width. Example: `@media (max-width: 768px) { /* styles for small screens */ }`. They are fundamental to responsive design.

Q25. What is flexbox?

Ans: Flexbox (CSS Flexible Box Layout) is a one-dimensional layout model that arranges items in rows or columns. Properties like `display:flex`, `justify-content`, `align-items`, `flex-wrap` enable flexible and responsive layouts easily.

Q26. What is grid layout?

Ans: CSS Grid Layout is a two-dimensional layout system that divides a page into rows and columns. With `display:grid`, `grid-template-columns`, `grid-template-rows`, and `grid-gap`, complex layouts can be created with minimal code.

Q27. What is specificity in CSS?

Ans: Specificity determines which CSS rule applies when multiple rules target the same element. Priority order: Inline styles > ID selectors > Class/attribute/pseudo-class > Element selectors. Calculated as (a, b, c, d) values.

Q28. What is z-index?

Ans: z-index controls the stacking order of positioned elements (non-static). Higher z-index values appear in front. It only works on elements with position set to relative, absolute, fixed, or sticky.

Q29. How to optimize webpage performance?

Ans: Optimization techniques: minify CSS/JS/HTML, compress images (WebP format), use lazy loading, enable browser caching, use CDN, reduce HTTP requests, enable GZIP compression, use async/defer for scripts, and leverage CSS sprites.

Q30. What are best practices in HTML and CSS design?

Ans: Best practices: use semantic HTML, external CSS files, consistent naming conventions (BEM), mobile-first approach, validate code (W3C Validator), maintain accessibility (ARIA), optimize performance, use CSS variables, and comment code.

Practical 3: XML + DTD + XML Schema

Q1. What is XML?

Ans: XML (eXtensible Markup Language) is a markup language designed to store and transport data in a structured, human-readable, and machine-readable format. Unlike HTML, XML tags are user-defined and focus on data rather than presentation.

Q2. What is the purpose of XML?

Ans: XML is used to store, transport, and exchange structured data between different systems and platforms in a format that is both human-readable and machine-readable, independent of software and hardware.

Q3. Difference between XML and HTML?

Ans: HTML defines fixed tags for web page presentation; XML uses custom user-defined tags for data storage/exchange. HTML is not strict about closing tags; XML requires well-formed documents. HTML is for display; XML is for data.

Q4. What is a tag in XML?

Ans: An XML tag is a markup label enclosed in angle brackets that defines elements. Tags come in pairs: <tagname> (start tag) and </tagname> (end tag), or as empty elements: <tagname/>. Tags are case-sensitive in XML.

Q5. What is an element?

Ans: An XML element consists of a start tag, content, and end tag. Example: <name>Farhan</name>. Elements can contain text, attributes, other elements, or be empty. The root element encloses all other elements.

Q6. What is an attribute in XML?

Ans: Attributes provide metadata about XML elements. They appear within the start tag as name-value pairs with quoted values. Example: <student roll='23CO011'>. Attributes should not contain structural data; elements are preferred for data.

Q7. What is well-formed XML?

Ans: A well-formed XML document follows syntactic rules: single root element, all tags properly closed, tags correctly nested, attribute values quoted, case-sensitive tag names, and no illegal characters. Well-formed XML can be parsed by any XML parser.

Q8. What is valid XML?

Ans: A valid XML document is both well-formed AND conforms to a specific structure defined by a DTD or XML Schema. Validity ensures the document has required elements, correct data types, and proper hierarchy.

Q9. What is DTD?

Ans: DTD (Document Type Definition) defines the legal structure of an XML document — which elements and attributes are allowed, their order, and whether they are required or optional. DTD uses its own syntax (not XML-based).

Q10. What is XML Schema?

Ans: XML Schema (XSD — XML Schema Definition) defines the structure, data types, and constraints for XML documents. Unlike DTD, it uses XML syntax, supports data typing, namespaces, and is more powerful and flexible.

Q11. Difference between DTD and XML Schema?

Ans: DTD: limited data typing, own non-XML syntax, no namespace support, simpler. XML Schema (XSD): rich data typing (string, int, date), written in XML, supports namespaces, more verbose but more powerful and widely used in industry.

Q12. What is XSL?

Ans: XSL (eXtensible Stylesheet Language) is a family of languages for transforming and rendering XML documents. It includes XSLT (transformation), XPath (addressing), and XSL-FO (formatting objects for print/display).

Q13. What is XSLT?

Ans: XSLT (XSL Transformations) is a language for transforming XML documents into other formats (HTML, plain text, another XML structure). It uses template rules to match elements and define their transformation output.

Q14. What is XPath?

Ans: XPath (XML Path Language) is a query language for navigating and selecting nodes in an XML document. It uses path expressions like /root/child/element. XPath is used by XSLT and is essential for XML processing.

Q15. What is namespace in XML?

Ans: An XML namespace avoids element name conflicts when combining XML from different sources. It is defined using xmlns attribute: <root xmlns:ns='http://example.com'>. Namespaced elements are prefixed: <ns:student>.

Q16. What is CDATA section?

Ans: CDATA (Character Data) section in XML tells the parser to treat the enclosed text as literal character data, not markup. Syntax: <![CDATA[... text with <, >, & ...]]>. Used when element content contains special characters.

Q17. What is XML parser?

Ans: An XML parser is a software program that reads an XML document and makes its data available to applications. It checks for well-formedness and optionally validates against DTD/Schema. Examples: Java's DOM, SAX, JAXB parsers.

Q18. Types of XML parsers?

Ans: Two main types: (1) Tree-based parsers (DOM) — load entire XML into memory as a tree; (2) Event-based parsers (SAX, StAX) — parse sequentially, triggering events for each element without loading entire document.

Q19. What is DOM parser?

Ans: DOM (Document Object Model) parser reads the entire XML document into memory as a tree structure. It allows random access and modification of any part of the document but consumes more memory, suitable for small-to-medium files.

Q20. What is SAX parser?

Ans: SAX (Simple API for XML) is an event-driven parser that reads XML sequentially and triggers callback methods (startElement, endElement, characters) for each element. It is memory-efficient but read-only and forward-only.

Q21. Difference between DOM and SAX?

Ans: DOM loads the entire document into memory (tree), allows random access and modification, but uses more memory. SAX reads sequentially, triggers events, uses less memory, is faster for large files, but allows no modification and no random access.

Q22. How to validate XML document?

Ans: XML can be validated against a DTD using `<!DOCTYPE>` declaration, or against an XML Schema (XSD) using the `schemaLocation` attribute. Validation tools include XMLSpy, oXygen, or built-in Java XML parsers with schema validation enabled.

Q23. What is schema validation?

Ans: Schema validation checks an XML document against an XSD (XML Schema Definition) to ensure it conforms to the defined structure, required elements, data types, and constraints. It is more robust than DTD validation.

Q24. What is data typing in XML Schema?

Ans: XML Schema supports built-in data types: string, integer, float, boolean, date, dateTime, anyURI, etc. Data types enforce that element/attribute values conform to expected formats, unlike DTD which treats all content as text.

Q25. What is complex type in XML Schema?

Ans: A complexType in XSD defines an element that can contain other elements and/or attributes. It uses compositors like `<xs:sequence>` (ordered), `<xs:all>` (any order), or `<xs:choice>` (one of) to define child element structures.

Q26. What is simple type?

Ans: A simpleType in XSD defines an element or attribute that contains only text content with no child elements. It can restrict built-in types using facets like `minLength`, `maxLength`, `pattern` (regex), `enumeration`, `minInclusive`, etc.

Q27. How XML is displayed using CSS?

Ans: XML can be styled and displayed in a browser by linking a CSS stylesheet using the processing instruction: `<?xml-stylesheet type='text/css' href='style.css'?>`. CSS selectors match XML element names and apply styles.

Q28. How XML is transformed using XSLT?

Ans: An XSLT stylesheet defines templates using `<xsl:template match='... '>` to match XML nodes and specify their transformation output (usually HTML). The XSLT processor applies these templates, traversing the XML tree to produce output.

Q29. Advantages of XML Schema over DTD?

Ans: XML Schema advantages: XML syntax (easier to learn), rich data type support, namespace awareness, more expressive constraints (min/max values, patterns), supports inheritance and complex structures, widely adopted in web services.

Q30. Applications of XML in real world?

Ans: XML is used in: web services (SOAP/WSDL), configuration files (Maven pom.xml, Android layout), office documents (DOCX/XLSX are ZIP+XML), RSS/Atom feeds, SVG graphics, database data exchange, and AJAX (XMLHttpRequest).

Practical 4: JavaScript Application (Calculator)

Q1. What is JavaScript?

Ans: JavaScript (JS) is a lightweight, interpreted, high-level, dynamic programming language primarily used for adding interactivity to web pages. It runs in the browser (client-side) and also on the server (Node.js). It is one of the core web technologies alongside HTML and CSS.

Q2. What is the role of JavaScript in web development?

Ans: JavaScript handles client-side behavior: DOM manipulation, form validation, event handling, animations, AJAX calls, and dynamic content updates. It enables interactive web applications (SPAs) and communicates with servers via APIs.

Q3. Difference between JavaScript and Java?

Ans: Java is a compiled, strongly typed, class-based OOP language (JVM-based). JavaScript is interpreted, weakly typed, prototype-based, and runs in browsers. Despite the similar name, they are entirely different languages with different use cases.

Q4. What are variables in JavaScript?

Ans: Variables are named storage containers for data. In JavaScript, variables are declared using `var` (function-scoped), `let` (block-scoped, reassignable), or `const` (block-scoped, not reassignable). Example: `let x = 10; const PI = 3.14;`

Q5. Difference between var, let, and const?

Ans: `var`: function-scoped, hoisted, can be redeclared (older, avoid using). `let`: block-scoped, not hoisted to usable state, can be reassigned, cannot be redeclared. `const`: block-scoped, must be initialized, cannot be reassigned (but object properties can change).

Q6. What are data types in JavaScript?

Ans: Primitive types: Number, String, Boolean, undefined, null, Symbol, BigInt. Reference types: Object, Array, Function. JavaScript is dynamically typed — variables can hold any type and types are determined at runtime.

Q7. What is a function?

Ans: A function is a reusable block of code that performs a specific task. Declared using function keyword or arrow syntax: `function add(a, b) { return a + b; }` or `const add = (a, b) => a + b;`. Functions can accept parameters and return values.

Q8. What are operators in JavaScript?

Ans: JavaScript operators: Arithmetic (+, -, *, /, %, **), Assignment (=, +=, -=), Comparison (==, ===, !=, !==, >, <), Logical (&&, ||, !), Bitwise (&, |, ^), Ternary (condition ? a : b), typeof, instanceof.

Q9. What is an event?

Ans: An event is an action or occurrence detected by the browser, such as user interactions (click, keypress, mouseover, submit, load, change) or browser actions (DOMContentLoaded). Events trigger code execution through event handlers.

Q10. What is event handling?

Ans: Event handling is the process of detecting events and executing code in response. Done via HTML attributes (`onclick='fn()'`), DOM property (`element.onclick = fn`), or `addEventListener()` method (recommended, allows multiple handlers).

Q11. What is DOM?

Ans: DOM (Document Object Model) is a programming interface for HTML/XML documents. It represents the page as a tree of objects (nodes). JavaScript uses the DOM to dynamically access, modify, add, or delete elements and attributes.

Q12. What is BOM?

Ans: BOM (Browser Object Model) represents the browser window and provides objects for interacting with it: window (global), navigator (browser info), location (URL), history (browsing history), screen (display info), document (DOM root).

Q13. What is alert() function?

Ans: alert() is a BOM method that displays a dialog box with a message and an OK button. It blocks execution until dismissed. Syntax: alert('message'). Used for simple notifications but considered poor UX for production applications.

Q14. What is prompt() function?

Ans: prompt() displays a dialog box that asks the user for input. It returns the entered string (or null if cancelled). Syntax: let input = prompt('Enter name:'); The input is always a string type.

Q15. What is confirm()?

Ans: confirm() displays a dialog box with a message and OK/Cancel buttons. It returns true (OK clicked) or false (Cancel clicked). Syntax: let result = confirm('Delete?'); Used for simple yes/no confirmations.

Q16. Difference between == and ===?

Ans: == (loose equality) compares values after type coercion: '5' == 5 returns true. === (strict equality) compares both value AND type without coercion: '5' === 5 returns false. Always prefer === to avoid unexpected bugs.

Q17. What is NaN?

Ans: NaN (Not a Number) is a special numeric value indicating an invalid arithmetic operation (e.g., dividing 0/0, or parseInt('abc')). typeof NaN === 'number'. Use isNaN() or Number.isNaN() to check for NaN values.

Q18. What is type coercion?

Ans: Type coercion is JavaScript's automatic conversion of values from one type to another. Implicit coercion: '5' + 3 = '53' (string concatenation). Explicit coercion: Number('5') = 5, String(5) = '5'. Can cause unexpected behavior.

Q19. What is input validation?

Ans: Input validation ensures user-entered data is correct, complete, and secure before processing. It prevents errors (NaN in calculations) and security issues (injection attacks). Can be client-side (JS) or server-side.

Q20. How to validate numeric input?

Ans: Use isNaN(value) to check if input is not a number. Also use isFinite(), or check value === "" for empty input. Example: if(isNaN(input) || input === "") { alert('Enter valid number'); }

Q21. What are conditional statements?

Ans: Conditional statements control program flow based on conditions: if-else (single/multiple conditions), else if (chained conditions), switch-case (multiple fixed values), ternary operator (condition ? a : b for inline conditionals).

Q22. What are loops?

Ans: Loops repeatedly execute a block of code: for (known iterations), while (condition-based), do-while (executes at least once), for...in (object properties), for...of (iterables like arrays). break exits a loop; continue skips an iteration.

Q23. What is a callback function?

Ans: A callback is a function passed as an argument to another function, to be executed later (after an operation completes). Example: setTimeout(function() { alert('Done'); }, 1000). Widely used in asynchronous JavaScript.

Q24. What is closure?

Ans: A closure is a function that retains access to its outer function's variables even after the outer function has returned. It creates a private scope. Example: `function counter() { let count = 0; return function() { return ++count; }; }`

Q25. What is event bubbling?

Ans: Event bubbling means when an event occurs on a child element, it propagates up through the DOM tree to parent elements, triggering their event handlers too. Use `event.stopPropagation()` to stop bubbling.

Q26. What is asynchronous JavaScript?

Ans: Asynchronous JS allows operations (API calls, file I/O, timers) to run without blocking the main thread. Implemented using: callbacks, Promises (`.then()/.catch()`), and `async/await` syntax for cleaner asynchronous code.

Q27. What is `setTimeout()`?

Ans: `setTimeout(callback, delay)` executes a function once after a specified delay in milliseconds. Example: `setTimeout(() => console.log('Hello'), 2000)`. Returns a timer ID that can be used with `clearTimeout()` to cancel.

Q28. What is `addEventListener()`?

Ans: `addEventListener(type, handler, useCapture)` attaches an event listener to an element. Multiple listeners can be added for the same event. It is the recommended way to handle events over inline handlers or DOM properties.

Q29. How to handle invalid input in calculator?

Ans: Check: (1) empty fields using `input === ''`, (2) non-numeric values using `isNaN(Number(input))`, (3) division by zero, (4) overflow/underflow. Display clear error messages without alerting and restore the calculator to a valid state.

Q30. What are common errors in JavaScript applications?

Ans: Common errors: `ReferenceError` (undefined variable), `TypeError` (wrong type operation), `SyntaxError` (invalid syntax), `RangeError` (value out of range), undefined errors, NaN propagation, scope issues, and asynchronous timing bugs.

Practical 5: Servlet Program

Q1. What is a servlet?

Ans: A servlet is a Java class that runs on a web server (inside a servlet container) to handle HTTP requests and generate dynamic web responses. It extends `HttpServlet` and overrides `doGet()` or `doPost()` methods.

Q2. What is a web server?

Ans: A web server is software that handles HTTP requests from clients (browsers) and serves static files or forwards requests to dynamic components (servlets). Examples: Apache HTTP Server, Nginx, Apache Tomcat.

Q3. What is a servlet container?

Ans: A servlet container (web container) is a component of an application server that manages the servlet lifecycle, handles HTTP communication, security, multithreading, and deployment. Example: Apache Tomcat, Jetty, WildFly.

Q4. What is HTTP?

Ans: HTTP (HyperText Transfer Protocol) is the stateless request-response protocol used for communication between web clients and servers. It defines methods (GET, POST, PUT, DELETE), status codes (200, 404, 500), and message structure.

Q5. What is a request and response?

Ans: An HTTP request is sent by the client containing method, URL, headers, and body (for POST). The server processes it and sends an HTTP response with status code, headers, and body (HTML, JSON, etc.).

Q6. What is JDBC?

Ans: JDBC (Java Database Connectivity) is a Java API that provides a standard interface for Java applications to connect to and interact with relational databases. Key classes: DriverManager, Connection, Statement, PreparedStatement, ResultSet.

Q7. What is a database?

Ans: A database is an organized collection of structured data stored electronically. Relational databases store data in tables (rows and columns) with relationships between tables. Examples: MySQL, Oracle, PostgreSQL.

Q8. What is a table?

Ans: A database table is a structured collection of data organized into rows (records) and columns (attributes/fields). Each table represents an entity (e.g., Student, Product). Tables are the fundamental storage unit in relational databases.

Q9. What is SQL?

Ans: SQL (Structured Query Language) is the standard language for managing relational databases. It includes: DML (SELECT, INSERT, UPDATE, DELETE), DDL (CREATE, ALTER, DROP), DCL (GRANT, REVOKE), and TCL (COMMIT, ROLLBACK).

Q10. What is SELECT query?

Ans: SELECT retrieves data from one or more tables. Syntax: SELECT column1, column2 FROM tablename WHERE condition ORDER BY column; SELECT * retrieves all columns. It can use JOINS, GROUP BY, HAVING, and aggregate functions.

Q11. What is servlet life cycle?

Ans: The servlet lifecycle: (1) Loading and instantiation — container loads the class; (2) init() — called once for initialization; (3) service() — called for each request (calls doGet/doPost); (4) destroy() — called before removal for cleanup.

Q12. What is init() method?

Ans: init(ServletConfig config) is called once when the servlet is first loaded. Used for one-time initialization: loading configuration, opening database connections, initializing resources. It takes a ServletConfig parameter.

Q13. What is service() method?

Ans: service(HttpServletRequest req, HttpServletResponse res) is called for every HTTP request. The container calls it, which internally calls doGet() for GET requests, doPost() for POST requests, etc. It handles request dispatching.

Q14. What is destroy() method?

Ans: destroy() is called once before the servlet is removed from service (container shutdown or redeployment). Used for cleanup: closing database connections, releasing resources, saving state. It is the counterpart of init().

Q15. Difference between GET and POST?

Ans: GET: appends data to URL (visible, bookmarkable), length-limited, cached, used for retrieval. POST: sends data in request body (not visible in URL), no size limit, not cached, used for submitting data that modifies server state.

Q16. What is session?

Ans: A session is a server-side mechanism to maintain state between multiple HTTP requests from the same user. The container creates an HttpSession object, stores user data (attributes), and identifies the session via a JSESSIONID cookie.

Q17. What is cookie?

Ans: A cookie is a small piece of data sent by the server to the browser, which stores it and sends it back with subsequent requests. Used for session tracking, remembering preferences, and login status. Cookies can be persistent or session-based.

Q18. What is session tracking?

Ans: Session tracking maintains user state across multiple HTTP requests (since HTTP is stateless). Techniques: Cookies (JSESSIONID), URL rewriting (appending session ID to URL), Hidden form fields, SSL sessions, and HttpSession API.

Q19. How servlet connects to database?

Ans: Steps: (1) Load JDBC driver: `Class.forName('com.mysql.cj.jdbc.Driver');` (2) Establish connection: `Connection con = DriverManager.getConnection(url, user, pass);` (3) Create Statement/PreparedStatement; (4) Execute query; (5) Close resources in finally.

Q20. What is driver manager?

Ans: DriverManager is a JDBC class that manages database drivers and creates database connections. `DriverManager.getConnection(url, username, password)` returns a Connection object. It acts as a factory for connections.

Q21. What is result set?

Ans: ResultSet is a JDBC interface representing the result of a database query. It holds the rows returned by SELECT statement. Use `rs.next()` to iterate rows and `rs.getString('column')`, `rs.getInt('column')` to retrieve values.

Q22. What is prepared statement?

Ans: PreparedStatement is a precompiled JDBC SQL statement that uses placeholders (?). Example: `PreparedStatement ps = con.prepareStatement('SELECT * FROM users WHERE id=?');` `ps.setInt(1, userId);` It prevents SQL injection and improves performance.

Q23. What is connection pooling?

Ans: Connection pooling maintains a pool of pre-established database connections that are reused for multiple requests, instead of creating new connections each time. It significantly improves performance. Libraries: Apache DBCP, HikariCP, C3P0.

Q24. What is servlet configuration?

Ans: Servlet configuration (ServletConfig) provides initialization parameters and the ServletContext to a servlet. Parameters are defined in web.xml or via `@WebServlet` annotation. Accessed via: `getInitParameter('paramName')`.

Q25. What is web.xml?

Ans: web.xml is the deployment descriptor file of a Java web application. It configures servlets (class mapping, URL mapping), filters, listeners, welcome files, error pages, session timeout, and security constraints.

Q26. What is servlet mapping?

Ans: Servlet mapping defines the URL pattern that triggers a specific servlet. Defined in web.xml via `<servlet-mapping>` or via `@WebServlet` annotation. Example: `@WebServlet('/login')` maps `/login` URL to the LoginServlet class.

Q27. What is request dispatcher?

Ans: RequestDispatcher allows a servlet to forward a request to another servlet, JSP, or HTML, or include another resource's output. Methods: `forward()` (transfers control) and `include()` (includes output). Obtained via `request.getRequestDispatcher('path')`.

Q28. What is filter in servlet?

Ans: A filter is a reusable Java class that intercepts HTTP requests and responses. Used for: authentication, logging, encoding, compression, input validation. Implements `javax.servlet.Filter` and configured in `web.xml` or via `@WebFilter` annotation.

Q29. What are security concerns in servlet?

Ans: Security concerns: SQL injection (use `PreparedStatement`), XSS (sanitize output), CSRF (use tokens), session fixation (regenerate session ID on login), sensitive data exposure (HTTPS), and improper error handling revealing server details.

Q30. What are advantages of servlet?

Ans: Advantages: platform-independent (Java), high performance (runs in server JVM), supports multithreading, access to full Java APIs, session/cookie management, robust error handling, integration with frameworks (Spring, Struts), and scalability.

Practical 6: JSP Program

Q1. What is JSP?

Ans: JSP (JavaServer Pages) is a Java technology for creating dynamic web content. It allows embedding Java code in HTML pages using scriptlets, expressions, and directives. JSP is converted to a servlet by the container before execution.

Q2. What is the purpose of JSP?

Ans: JSP simplifies the creation of dynamic web pages by allowing developers to mix HTML markup with Java code. It separates presentation logic from business logic, making web application development more maintainable.

Q3. Difference between JSP and servlet?

Ans: Servlet is a Java class for request handling (business logic); JSP is HTML with embedded Java (presentation layer). JSP is easier for UI design; Servlet is better for logic. JSP is converted to a Servlet internally. MVC: Servlet = Controller, JSP = View.

Q4. What is JSP life cycle?

Ans: JSP Life Cycle: (1) Translation — JSP is converted to a servlet .java file; (2) Compilation — compiled to .class file; (3) Loading — class loaded into JVM; (4) Instantiation; (5) `jspInit()` — initialization; (6) `_jspService()` — called for each request; (7) `jspDestroy()` — cleanup.

Q5. What is a directive?

Ans: JSP directives provide global instructions to the JSP container about how to process the page. They affect the overall structure of the servlet generated from JSP. Written as `<%@ directive attribute='value' %>`.

Q6. Types of JSP directives?

Ans: Three types: (1) page directive (`<%@ page %>`) — page settings like import, contentType, session, errorPage; (2) include directive (`<%@ include %>`) — statically includes a file at compile time; (3) taglib directive (`<%@ taglib %>`) — declares a custom tag library.

Q7. What is scriptlet?

Ans: A scriptlet embeds Java code directly inside a JSP page. Written within `<% ... %>` tags. The code runs within the `_jspService()` method. Example: `<% String name = request.getParameter('name'); %>`. Excessive scriptlets are bad practice (use JSTL instead).

Q8. What is expression in JSP?

Ans: A JSP expression evaluates a Java expression and converts the result to a String for output. Written within `<%= ... %>` tags (no semicolon). Example: `<%= new java.util.Date() %>` outputs the current date. It calls `out.print()` internally.

Q9. What is declaration tag?

Ans: A declaration tag (`<%! ... %>`) declares variables and methods that become instance variables/methods of the generated servlet class. Unlike scriptlets (method-level), declarations are class-level. Example: `<%! int count = 0; %>`

Q10. What are implicit objects?

Ans: Implicit objects are pre-created objects available in JSP without explicit declaration. The JSP container creates them automatically. They provide access to request, response, session, application, output, and other web context objects.

Q11. Name JSP implicit objects.

Ans: Nine implicit objects: request (`HttpServletRequest`), response (`HttpServletResponse`), session (`HttpSession`), application (`ServletContext`), out (`JspWriter`), config (`ServletConfig`), pageContext (`PageContext`), page (current servlet instance), exception (`Throwable`, for error pages).

Q12. What is request object?

Ans: The request implicit object (`HttpServletRequest`) represents the HTTP request. Used to retrieve form parameters (`getParameter()`), request headers (`getHeader()`), cookies (`getCookies()`), URL info, and request attributes (`getAttribute()/setAttribute()`).

Q13. What is response object?

Ans: The response implicit object (`HttpServletResponse`) represents the HTTP response. Used to set content type (`setContentType()`), send redirects (`sendRedirect()`), set headers (`setHeader()`), add cookies (`addCookie()`), and set status codes.

Q14. What is session object?

Ans: The session implicit object (`HttpSession`) maintains user state across multiple requests. Used to store user data: `session.setAttribute('user', userName); session.getAttribute('user');` `session.invalidate()` to log out. Session timeout is configurable.

Q15. What is application object?

Ans: The application implicit object (`ServletContext`) represents the entire web application. It is shared across all servlets and JSPs. Used for application-wide data (`setAttribute/getAttribute`), logging, accessing web app resources, and reading init parameters.

Q16. What is JSP action tag?

Ans: JSP action tags control the behavior of the servlet engine. They use XML syntax and are executed at request time. Standard action tags include `<jsp:include>`, `<jsp:forward>`, `<jsp:useBean>`, `<jsp:setProperty>`, `<jsp:getProperty>`, `<jsp:param>`.

Q17. What is `<jsp:include>`?

Ans: `<jsp:include page='file.jsp'/>` dynamically includes a resource (JSP, HTML, servlet) at request time. Unlike the include directive (compile-time), it includes the response of the included page. Used for headers, footers, and reusable page fragments.

Q18. What is `<jsp:forward>`?

Ans: `<jsp:forward page='target.jsp'/>` transfers control from the current JSP to another resource (JSP, servlet, HTML) on the server. The URL in the browser does not change. Request attributes can be passed using `<jsp:param>` nested inside.

Q19. What is expression language (EL)?

Ans: EL (Expression Language) simplifies access to data stored in JavaBeans, request/session/application attributes, and request parameters using `${...}` syntax. Example: `${user.name}` accesses name property of user bean. Reduces scriptlet code significantly.

Q20. What is JSTL?

Ans: JSTL (JavaServer Pages Standard Tag Library) provides a collection of standard tags for common JSP tasks: `<c:forEach>` (iteration), `<c:if>`, `<c:choose>` (conditionals), `<c:set>`, `<c:out>` (output), `<sql:query>` (database), `<fmt:formatDate>`. Replaces scriptlets.

Q21. What is MVC architecture?

Ans: MVC (Model-View-Controller): Model — business logic and data (Java Beans, EJB); View — presentation layer (JSP, HTML); Controller — request handling and flow control (Servlet). Separates concerns for maintainability and testability.

Q22. How JSP is converted into servlet?

Ans: The JSP container translates the JSP file into a Java servlet source (.java), compiles it to a .class file, loads it, and executes it like a regular servlet. Directives, scriptlets, and expressions are translated into appropriate Java code in `_jspService()`.

Q23. What is custom tag?

Ans: Custom tags (user-defined tags) extend JSP with reusable, encapsulated functionality using tag libraries. They are defined by implementing Tag, SimpleTag, or BodyTag interfaces and described in a TLD (Tag Library Descriptor) file.

Q24. What is error handling in JSP?

Ans: JSP error handling: (1) `errorPage` directive — `<%@ page errorPage='error.jsp' %>` — redirects exceptions to an error page; (2) `isErrorPage='true'` on the error page enables access to the exception implicit object; (3) web.xml `<error-page>` configuration.

Q25. What is session management?

Ans: Session management maintains user state across multiple HTTP requests (stateless protocol). In JSP, using the session implicit object: `session.setAttribute()`, `session.getAttribute()`, `session.invalidate()`. Also uses cookies, URL rewriting, and hidden fields.

Q26. What is page directive?

Ans: The page directive `<%@ page ... %>` sets page-level properties: `import` (Java imports), `contentType` (MIME type), `pageEncoding`, `session` (true/false), `buffer` (output buffer size), `errorPage` (error JSP), `isErrorPage`, `autoFlush`, and `extends`.

Q27. What is include directive?

Ans: `<%@ include file='header.html' %>` statically includes the content of another file at translation time (compile time). The included file's content is merged into the current JSP before compilation. Changes in the included file require recompilation.

Q28. What is tag library?

Ans: A tag library is a collection of custom JSP tags packaged in a JAR file with a TLD (Tag Library Descriptor) XML file. The `taglib` directive `<%@ taglib uri='...' prefix='c' %>` imports it. JSTL is the most widely used standard tag library.

Q29. What are advantages of JSP?

Ans: Advantages: easy HTML + Java integration, implicit objects simplify coding, automatic session management, platform-independent, support for custom tags, EL and JSTL reduce scriptlet code, integration with Servlet/MVC frameworks, and good tool support.

Q30. What are limitations of JSP?

Ans: Limitations: mixing HTML and Java code reduces readability, difficult to debug translated servlet code, performance overhead on first request (translation + compilation), not ideal for complex business logic, learning curve for JSTL/EL, and memory usage.

Practical 7: PHP + MySQL Application

Q1. What is PHP?

Ans: PHP (Hypertext Preprocessor) is a server-side, open-source scripting language designed for web development. PHP code is executed on the server, generating HTML dynamically. It supports databases, sessions, file handling, and OOP.

Q2. What is MySQL?

Ans: MySQL is a popular open-source relational database management system (RDBMS) that uses SQL for data management. It is widely used with PHP for web applications (LAMP stack). It supports transactions, triggers, stored procedures, and replication.

Q3. What is a database?

Ans: A database is an organized collection of structured data stored and managed electronically. Relational databases store data in tables with rows and columns and support SQL queries. MySQL organizes data into databases containing tables.

Q4. What is a table?

Ans: A MySQL table stores data in rows (records) and columns (fields/attributes). Each table has a defined schema (column names, data types, constraints). Example: `CREATE TABLE students (id INT, name VARCHAR(50), marks FLOAT);`

Q5. What is primary key?

Ans: A primary key is a column (or combination of columns) that uniquely identifies each row in a table. It must be NOT NULL and unique. Used to enforce entity integrity and establish relationships between tables. Only one primary key per table.

Q6. What is foreign key?

Ans: A foreign key is a column in one table that references the primary key of another table, establishing a relationship between them. It enforces referential integrity — values must exist in the referenced table. Example: `student_id` in marks table referencing students table.

Q7. What is CRUD operation?

Ans: CRUD stands for the four basic database operations: Create (INSERT), Read (SELECT), Update (UPDATE), Delete (DELETE). Most web applications are built around CRUD operations on database entities.

Q8. What is SQL?

Ans: SQL (Structured Query Language) is the standard language for interacting with relational databases. It includes DML (SELECT, INSERT, UPDATE, DELETE), DDL (CREATE, ALTER, DROP, TRUNCATE), DCL (GRANT, REVOKE), and TCL (COMMIT, ROLLBACK).

Q9. What is SELECT query?

Ans: SELECT retrieves data from tables. Syntax: `SELECT column1, column2 FROM table WHERE condition ORDER BY col LIMIT n;` Supports JOIN (combine tables), GROUP BY (aggregate), HAVING (filter groups), and subqueries.

Q10. What is INSERT query?

Ans: INSERT adds new rows to a table. Syntax: `INSERT INTO tablename (col1, col2) VALUES (val1, val2);` To insert multiple rows: `INSERT INTO table (col1) VALUES (v1), (v2), (v3);` In PHP, use prepared statements to prevent SQL injection.

Q11. What is UPDATE query?

Ans: UPDATE modifies existing rows. Syntax: `UPDATE tablename SET col1=val1, col2=val2 WHERE condition;` Always use WHERE clause to avoid updating all rows. Example: `UPDATE students SET marks=90 WHERE id=5;`

Q12. What is DELETE query?

Ans: DELETE removes rows from a table. Syntax: `DELETE FROM tablename WHERE condition;` Without WHERE, all rows are deleted. TRUNCATE TABLE is faster for deleting all rows but cannot be rolled back. Use prepared statements in PHP.

Q13. How PHP connects to MySQL?

Ans: Two ways: (1) mysqli (object-oriented): `$conn = new mysqli('localhost','user','pass','db');`
 (2) PDO: `$pdo = new PDO('mysql:host=localhost;dbname=db', 'user', 'pass');` Check connection, query, fetch results, and close connection.

Q14. What is mysqli?

Ans: mysqli (MySQL Improved) is a PHP extension for connecting to MySQL databases. It offers two interfaces: procedural (`mysqli_connect()`) and OOP (`new mysqli()`). It supports prepared statements, transactions, and multiple queries. Successor to the deprecated mysql extension.

Q15. What is PDO?

Ans: PDO (PHP Data Objects) is a PHP database abstraction layer that provides a uniform interface for multiple databases (MySQL, PostgreSQL, SQLite, etc.). It supports prepared statements natively, making it more secure and portable than mysqli.

Q16. Difference between GET and POST?

Ans: GET: data appended to URL (visible, bookmarkable, cached), used for retrieving data, limited by URL length (~2048 chars). POST: data in request body (hidden, not cached), used for submitting sensitive/large data, no size limit. Forms use `method='post'` for security.

Q17. What is session in PHP?

Ans: PHP sessions store user data server-side across multiple page requests. `session_start()` initiates a session; `$_SESSION['key'] = 'value'` stores data; `unset($_SESSION['key'])` removes data; `session_destroy()` ends session. Identified by a PHPSESSID cookie.

Q18. What is cookie?

Ans: Cookies store small data in the user's browser. In PHP: `setcookie('name', 'value', time()+3600)` sets a cookie valid for 1 hour; `$_COOKIE['name']` reads it. Cookies are sent with every HTTP request. Size limit ~4KB.

Q19. What is SQL injection?

Ans: SQL injection is a security attack where malicious SQL code is inserted into input fields to manipulate database queries. Example: entering ' OR '1'='1 in a login form. Can expose/delete all data. Prevented by using prepared statements and input validation.

Q20. How to prevent SQL injection?

Ans: Prevention methods: (1) Use prepared statements with parameter binding (`$stmt->bind_param()`); (2) Use PDO with prepared statements; (3) Validate and sanitize all user input; (4) Use `mysqli_real_escape_string()` (less preferred); (5) Apply principle of least privilege for DB users.

Q21. What is prepared statement?

Ans: A prepared statement separates SQL structure from data using placeholders (?). The query is sent to MySQL once; parameters are bound separately. This prevents SQL injection. Example: `$stmt = $conn->prepare('SELECT * FROM users WHERE email=?');` `$stmt->bind_param('s', $email);`

Q22. What is normalization?

Ans: Normalization is the process of organizing database tables to reduce data redundancy and improve data integrity. Normal Forms: 1NF (atomic values, no repeating groups), 2NF (no partial dependency), 3NF (no transitive dependency), BCNF.

Q23. What is indexing?

Ans: An index is a database structure that speeds up data retrieval. Like a book index, it allows MySQL to find rows without scanning the entire table. Types: Primary (auto on PK), Unique, Full-text. `CREATE INDEX idx_name ON table(column)`. Trade-off: faster reads, slower writes.

Q24. What is transaction?

Ans: A database transaction is a sequence of SQL operations executed as a single logical unit of work. It follows ACID properties. In MySQL: START TRANSACTION; (operations); COMMIT; or ROLLBACK; if an error occurs. Uses InnoDB engine.

Q25. What is commit and rollback?

Ans: COMMIT permanently saves all changes made in the current transaction to the database. ROLLBACK undoes all changes made in the current transaction if an error occurs, restoring the database to its previous state. Both are TCL commands in SQL.

Q26. What is error handling in PHP?

Ans: PHP error handling: die()/exit() for fatal errors, try-catch for exceptions, set_error_handler() for custom error handlers, error_reporting() to control displayed errors. mysqli: \$conn->connect_error checks connection; PDO throws PDOException.

Q27. What is form validation?

Ans: Form validation checks user input for correctness before processing. PHP validation: isset(), empty(), is_numeric(), filter_var(\$email, FILTER_VALIDATE_EMAIL), strlen() for length. Always validate server-side even if client-side JS validation exists.

Q28. What is file handling in PHP?

Ans: PHP file functions: fopen()/fclose() (open/close), fread()/fgets() (read), fwrite()/fputs() (write), file_get_contents()/file_put_contents() (entire file), unlink() (delete), file_exists(), is_file(), mkdir(), rename(). File permissions must be set correctly.

Q29. What are advantages of PHP?

Ans: Advantages: free and open-source, easy to learn, cross-platform (runs on Windows/Linux/Mac), vast community and documentation, direct MySQL integration, supports OOP, extensive libraries/frameworks (Laravel, CodeIgniter), embedded in HTML, and rapid development.

Q30. What are limitations of PHP?

Ans: Limitations: inconsistent function naming/parameter order, weak type system (before PHP 7), security vulnerabilities if misused (SQL injection, XSS), poor for desktop applications, memory management issues at scale, not suitable for CPU-intensive tasks.

Practical 8: Struts Login Validation

Q1. What is Struts framework?

Ans: Apache Struts is an open-source Java web framework that implements the MVC design pattern. It provides reusable components (Action, ActionForm, ActionMapping) for building enterprise Java web applications, handling request routing, validation, and views.

Q2. What is MVC architecture?

Ans: MVC (Model-View-Controller) is a design pattern separating application into three components: Model (data/business logic — Java Beans), View (presentation — JSP/HTML), Controller (request handler and coordinator — Servlet/Action class). Improves maintainability.

Q3. What is a model in MVC?

Ans: The Model represents the application's data and business logic. In Struts, it includes Java Beans, EJBs, or POJO classes. The model retrieves data from databases, processes business rules, and returns results to the controller.

Q4. What is a view in MVC?

Ans: The View is responsible for rendering data to the user interface. In Struts, JSP pages act as views. They receive data from the model (through request/session scope) and present it as HTML, using JSTL and EL to display dynamic content.

Q5. What is a controller in MVC?

Ans: The Controller handles user requests, invokes appropriate model operations, and determines which view to display. In Struts 1, the `ActionServlet` is the central controller (Front Controller pattern) that delegates to specific Action classes.

Q6. What is a form bean?

Ans: In Struts 1, a form bean (`ActionForm` subclass) is a Java class that holds form field data submitted by users. It maps HTML form fields to Java properties and can perform basic form validation. Configured in `struts-config.xml`.

Q7. What is Action class?

Ans: The Action class in Struts contains the business logic for a specific user request. It extends `Action` and implements the `execute()` method. It processes form data, interacts with the model, and returns an `ActionForward` to indicate the next view.

Q8. What is validation?

Ans: Validation is the process of checking that user input meets required criteria before processing. In Struts, validation can be done in the `ActionForm`'s `validate()` method or using the Validator framework with `validation.xml` configuration.

Q9. Why validation is required?

Ans: Validation ensures data integrity (correct format), security (prevents attacks), user experience (helpful error messages), and business rule compliance. Without validation, applications are vulnerable to bad data, SQL injection, and unexpected behavior.

Q10. What is client-side validation?

Ans: Client-side validation runs in the browser using JavaScript before form submission. It provides instant feedback to users without server round-trips. In Struts, client-side validation is generated automatically by the Validator framework using JavaScript.

Q11. What is server-side validation?

Ans: Server-side validation runs on the server after form submission. It is mandatory because client-side can be bypassed. In Struts, implemented in `ActionForm.validate()` method or Validator framework. Errors are added to `ActionErrors` and displayed in JSP.

Q12. How to validate name field?

Ans: Name validation: (1) Check not empty: `if(name == null || name.trim().isEmpty())` add error; (2) Check length: `if(name.length() < 2 || name.length() > 50)` add error; (3) Check format: `name.matches("[a-zA-Z ]+")` for letters only.

Q13. How to validate mobile number?

Ans: Mobile validation using regex: `if(!mobile.matches("[6-9][0-9]{9}"))` — valid Indian mobile starts with 6-9 and has exactly 10 digits. Also check: `!mobile.isEmpty()` and `mobile.length() == 10`. Example pattern: `^[6-9]\d{9}$`

Q14. How to validate email ID?

Ans: Email validation using regex: `if(!email.matches("[a-zA-Z0-9+_.-]+@[a-zA-Z0-9.-]+"))` for basic check. Or use Apache Commons Validator: `EmailValidator.getInstance().isValid(email)`. Check for @, domain, and proper format.

Q15. What is regular expression?

Ans: A regular expression (regex) is a pattern used to match character combinations in strings. In Java, `Pattern` and `Matcher` classes handle regex. Example: `[A-Za-z]{3,}` matches 3+ letters. Used for email, phone, password validation.

Q16. What happens if validation fails?

Ans: If `validate()` returns a non-empty `ActionErrors` object, Struts redirects back to the input page (specified by input attribute in `ActionMapping`) with error messages. The form fields retain their values and errors display using `<html:errors/>` tag.

Q17. What is error message handling?

Ans: In Struts, errors are created as `ActionMessage` objects added to `ActionErrors`. These are stored in request scope. JSP displays them using `<html:errors/>` tag or `<html:errors property='fieldName'/>`. Messages are defined in `ApplicationResources.properties` file.

Q18. What is Struts configuration file?

Ans: `struts-config.xml` is the central configuration file for a Struts 1 application. It defines: form beans (`<form-beans>`), global exceptions, global forwards, action mappings (`<action-mappings>`), controller settings, and message resources location.

Q19. What is struts-config.xml?

Ans: `struts-config.xml` maps URL paths to Action classes and defines input/forward pages. Example: `<action path='/login' type='LoginAction' name='loginForm' input='/login.jsp'> <forward name='success' path='/home.jsp'/> </action>`

Q20. What is ActionForm?

Ans: `ActionForm` is a JavaBean class (extends `org.apache.struts.action.ActionForm`) that encapsulates HTML form data. It has getter/setter methods for each form field. The container populates it automatically from request parameters before calling the Action.

Q21. What is ActionMapping?

Ans: `ActionMapping` contains configuration information for an Action, including: path (URL), type (Action class), name (form bean), scope, input (error page), and forwards (possible output pages). Defined in `struts-config.xml`'s `<action-mappings>`.

Q22. What is ActionForward?

Ans: `ActionForward` represents a logical destination (JSP or URL) to forward control to after an Action executes. The `Action.execute()` method returns an `ActionForward`. The `mapping.findForward('success')` looks up the forward by name from `struts-config.xml`.

Q23. What is session management?

Ans: In Struts, session management uses `HttpSession` (accessed via `request.getSession()`). User details (e.g., logged-in user) are stored in session: `session.setAttribute('user', user)`. Session data persists across requests until `session.invalidate()` is called.

Q24. What is token mechanism?

Ans: Struts provides a token mechanism (`saveToken(request)/isTokenValid(request)`) to prevent duplicate form submissions. A unique token is stored in session and embedded in form. On submit, the token is validated and invalidated to prevent resubmission.

Q25. What is form submission flow?

Ans: Flow: User fills form → POST to `ActionServlet` → `ActionServlet` populates `ActionForm` → `validate()` called (if errors, return to input JSP) → `Action.execute()` called → returns `ActionForward` → JSP rendered to user.

Q26. What are security issues in login forms?

Ans: Security issues: SQL injection (use `PreparedStatement`), brute force (account lockout, CAPTCHA), session fixation (regenerate session after login), XSS in error messages (sanitize output), password storage (hash with `BCrypt`), and HTTPS for credential transmission.

Q27. What is authentication?

Ans: Authentication verifies the identity of a user (who are you?). In web apps, typically done by checking username/password credentials against the database. After successful authentication, user identity is stored in session.

Q28. What is authorization?

Ans: Authorization determines what resources a verified user is allowed to access (what can you do?). Implemented after authentication using role-based access control (RBAC). In Struts, can be implemented via interceptors or filters.

Q29. Difference between Struts 1 and Struts 2?

Ans: Struts 1: ActionForm for form beans, XML-heavy configuration, ActionServlet as controller, limited AJAX support. Struts 2: POJO actions (no ActionForm), interceptor-based, annotation support, better AJAX/REST support, based on WebWork, cleaner architecture.

Q30. What are advantages of Struts?

Ans: Advantages: MVC separation, centralized configuration, built-in validation framework, tag libraries (html, bean, logic), form bean reuse, good integration with Spring/Hibernate, mature framework with large community, and enterprise-grade reliability.

Practical 9: AngularJS Application

Q1. What is AngularJS?

Ans: AngularJS is an open-source JavaScript framework by Google (released 2010) for building dynamic Single Page Applications (SPAs). It extends HTML with directives, implements MVC/MVVM architecture, provides two-way data binding, dependency injection, and routing.

Q2. What is a module?

Ans: An AngularJS module is a container for different application components (controllers, services, filters, directives). Created with: `var app = angular.module('myApp', []);` The array contains dependency module names. `ng-app='myApp'` bootstraps the module.

Q3. What is a controller?

Ans: A controller in AngularJS is a JavaScript constructor function that controls the data and behavior for a part of the page (defined by `ng-controller`). It sets up the `$scope` object with model data and functions. Created with: `app.controller('MyCtrl', function($scope){})`.

Q4. What is a directive?

Ans: Directives extend HTML with custom attributes and elements. Built-in directives: `ng-app`, `ng-controller`, `ng-model`, `ng-bind`, `ng-repeat`, `ng-if`, `ng-show`, `ng-hide`, `ng-click`, `ng-class`. Custom directives can encapsulate reusable UI behavior.

Q5. What is data binding?

Ans: Data binding creates a connection between the model (JavaScript) and the view (HTML), so changes in one automatically reflect in the other. AngularJS uses `{{ expression }}` for one-way binding (model to view) and `ng-model` for two-way binding.

Q6. What is two-way data binding?

Ans: Two-way data binding (`ng-model`) synchronizes the model and view automatically. When the user changes input (view), the model updates; when JavaScript changes the model, the view updates instantly. No manual DOM manipulation required.

Q7. What is ng-model?

Ans: `ng-model` is a directive that binds an HTML form element (input, select, textarea) to a `$scope` variable, creating two-way data binding. Example: `<input ng-model='username'>` — changing the input updates `$scope.username` and vice versa.

Q8. What is ng-app?

Ans: `ng-app` is a directive that bootstraps (initializes) an AngularJS application. Placed on the root HTML element (usually `<html>` or `<body>`). The value specifies the module name: `<div ng-app='myApp'>`. Angular processes all directives within this element.

Q9. What is ng-controller?

Ans: ng-controller attaches a controller to a DOM element, making its \$scope properties and methods available within that element. Example: `<div ng-controller='UserCtrl'>{{ username }}</div>`. Multiple controllers can exist on the same page.

Q10. What is ng-repeat?

Ans: ng-repeat iterates over a collection (array or object) and creates a copy of the element for each item. Example: `<li ng-repeat='item in items'>{{ item.name }}</li>`. Provides special variables: \$index, \$first, \$last, \$even, \$odd.

Q11. What is dependency injection?

Ans: DI is a design pattern where dependencies are provided to components rather than components creating them. AngularJS has a built-in DI container. Controllers, services, etc., declare dependencies as parameters: `app.controller('Ctrl', function($scope, $http){})`.

Q12. What is a service?

Ans: A service in AngularJS is a singleton object providing shared functionality across controllers. Created with `app.service('MyService', function() { this.method = function(){} })`. Built-in services: \$http, \$q, \$timeout, \$window, \$location, \$rootScope.

Q13. What is factory?

Ans: A factory is similar to a service but uses the factory pattern — returns an object instead of using 'this'. `app.factory('MyFactory', function() { return { method: function(){} }; })`. Factories allow better control over object creation and can return any value.

Q14. What is filter?

Ans: Filters transform displayed data in templates. Built-in filters: currency (`{{price | currency}}`), date, uppercase, lowercase, number, limitTo, orderBy, filter (search). Custom filters: `app.filter('myFilter', function(){ return function(input){...}; })`

Q15. What is routing?

Ans: Routing in AngularJS (ngRoute module) enables navigation between views in a SPA without full page reloads. `$routeProvider.when('/home', {templateUrl: 'home.html', controller: 'HomeCtrl'}).otherwise({redirectTo: '/home'})`; Uses `<ng-view>` as view container.

Q16. What is SPA (Single Page Application)?

Ans: SPA loads a single HTML page and dynamically updates content as the user interacts, without full page reloads. AngularJS enables SPAs using routing (ngRoute), dynamic content loading, and AJAX calls. Examples: Gmail, Google Maps, Facebook.

Q17. What is digest cycle?

Ans: The digest cycle is AngularJS's change detection mechanism. It compares current scope values with previous values (dirty checking). If changes are detected, it updates the view and runs again until no changes occur (stable state). Triggered by user actions or `$scope.$apply()`.

Q18. What is scope?

Ans: \$scope is a JavaScript object that acts as the glue between the controller and the view. It stores model data and functions. \$scope properties are accessible in the HTML template. Each ng-controller creates a new \$scope inheriting from \$rootScope.

Q19. What is \$rootScope?

Ans: \$rootScope is the top-level scope — the parent of all other scopes in an AngularJS application. Data on \$rootScope is accessible from all controllers and views. Used for application-wide data, but should be used sparingly to avoid namespace pollution.

Q20. What is form validation in AngularJS?

Ans: AngularJS provides automatic form validation using HTML5 attributes (required, minlength, maxlength, pattern, email) with ng-model. Form and field states: \$valid, \$invalid, \$dirty, \$pristine, \$touched. Display errors: `<span ng-show='form.field.$invalid'>Error</span>`

Q21. What is HTTP service?

Ans: \$http service makes AJAX calls to remote servers. Example: `$http.get('/api/users').then(function(response){ $scope.users = response.data; })`. Methods: `$http.get()`, `$http.post()`, `$http.put()`, `$http.delete()`. Returns a promise.

Q22. What is AJAX?

Ans: AJAX (Asynchronous JavaScript and XML) allows web pages to communicate with servers asynchronously without full page reloads. In AngularJS, \$http service handles AJAX. Modern APIs return JSON. AJAX enables dynamic data loading in SPAs.

Q23. What is promise?

Ans: A promise is an object representing a future asynchronous operation. \$http returns a promise with `.then(successFn, errorFn)` and `.catch(errorFn)`. AngularJS uses \$q service for creating custom promises. Promises avoid callback hell.

Q24. What is JSON?

Ans: JSON (JavaScript Object Notation) is a lightweight data interchange format based on JavaScript object syntax. Easy to read/write for humans and machines. In AngularJS, \$http API calls typically exchange data as JSON. Parsed with `JSON.parse()`; serialized with `JSON.stringify()`.

Q25. Difference between AngularJS and Angular?

Ans: AngularJS (v1.x): JavaScript, MVC, controller+\$scope pattern, older. Angular (v2+): TypeScript, component-based architecture, RxJS observables, better performance, mobile-friendly, different syntax. Angular 2+ is a complete rewrite, not backward compatible with AngularJS.

Q26. What is MVC in AngularJS?

Ans: AngularJS implements MVC: Model — \$scope data and services (data layer); View — HTML templates with directives and expressions; Controller — JavaScript code managing \$scope (logic layer). Some consider it MVVM because of two-way binding (ViewModel behavior).

Q27. What is directive lifecycle?

Ans: Custom directive lifecycle: `compile()` (processes DOM before scope linking, once), `pre-link()`, and `post-link()` (access scope and DOM, after compilation). The link function handles DOM manipulation and event binding. Compile is used for template transformation.

Q28. What is performance optimization?

Ans: AngularJS performance tips: limit the number of watchers (fewer ng-repeat items), use `track by` in ng-repeat, use one-time binding (`::value`), lazy loading, avoid heavy computations in `$watch`, use `ng-if` instead of `ng-show` for invisible elements, debounce input.

Q29. What is authentication in Angular apps?

Ans: AngularJS authentication: store JWT token in `sessionStorage`/`localStorage` after login; use \$http interceptors to attach Authorization header to all requests; check token on route change using \$routeProvider's `resolve`; logout by clearing token and redirecting.

Q30. What are advantages of AngularJS?

Ans: Advantages: two-way data binding reduces boilerplate, dependency injection promotes modular code, directives create reusable components, built-in form validation, good for SPAs, large ecosystem, Google support, declarative UI, and separation of concerns.

Practical 10: EJB Business Logic

Q1. What is EJB?

Ans: EJB (Enterprise JavaBeans) is a server-side component architecture for building scalable, distributed, and transactional enterprise Java applications. EJBs run inside an EJB container (part of a Java EE application server) that provides services like transactions, security, and concurrency.

Q2. What is enterprise application?

Ans: An enterprise application is a large-scale, multi-tiered software system designed for organizational use, requiring high availability, scalability, security, and integration with other systems. Examples: banking systems, ERP (SAP), healthcare management systems.

Q3. What is business logic?

Ans: Business logic is the part of an application that implements the specific rules, processes, and workflows of a business domain. In EJB, session beans encapsulate business logic, keeping it separate from presentation (JSP) and data (JPA entities).

Q4. What is middleware?

Ans: Middleware is software that acts as a bridge between different systems or application components. In Java EE, the application server acts as middleware, providing services (transactions, security, messaging, naming) that EJBs rely on.

Q5. What is application server?

Ans: An application server provides a runtime environment for Java EE applications, offering services: EJB container, Servlet/JSP container, JMS messaging, JNDI naming, JTA transactions, security, connection pooling, and clustering. Examples: JBoss, GlassFish, WebSphere, WebLogic.

Q6. What are types of EJB?

Ans: Three types: (1) Session Beans — implement business logic (Stateless, Stateful, Singleton); (2) Entity Beans (replaced by JPA entities in EJB 3.x) — represent persistent data; (3) Message-Driven Beans (MDB) — process asynchronous JMS messages.

Q7. What is session bean?

Ans: A session bean implements the business logic of an application. It is called by client components (servlets, JSPs). It can be Stateless (no client state between calls), Stateful (maintains state for one client), or Singleton (shared among all clients).

Q8. Types of session beans?

Ans: Three types: (1) Stateless Session Bean — no instance variables storing per-client state, pooled by container, high performance; (2) Stateful Session Bean — maintains conversational state with a specific client; (3) Singleton Session Bean — one instance per application, shared.

Q9. What is stateless session bean?

Ans: A stateless session bean does not maintain client-specific state between method calls. Each call is independent. The container maintains a pool of instances, assigning one for each request. Annotated with `@Stateless`. Good for tasks like calculations, database operations.

Q10. What is stateful session bean?

Ans: A stateful session bean maintains conversational state with a specific client across multiple method calls within a session. One instance per client. Annotated with `@Stateful`. Used for shopping carts, wizards, or multi-step transactions.

Q11. What is singleton bean?

Ans: A singleton session bean is instantiated once per application and shared among all clients. Annotated with `@Singleton`. Supports concurrent access (configured with

@ConcurrencyManagement). Used for application-wide caches, configuration data, startup initialization.

Q12. What is entity bean?

Ans: In older EJB versions (EJB 2.x), entity beans represented persistent data in a database. EJB 3.x replaced them with JPA (Java Persistence API) entities — simple POJO classes annotated with @Entity. JPA entities are managed by the EntityManager.

Q13. What is message-driven bean?

Ans: A Message-Driven Bean (MDB) is an EJB that processes asynchronous messages from a JMS (Java Message Service) queue or topic. It implements the MessageListener interface and the onMessage() method. Annotated with @MessageDriven. Used for asynchronous processing.

Q14. What is transaction?

Ans: A transaction is a unit of work that is executed atomically — all operations succeed or all fail together (ACID properties). EJB containers can manage transactions automatically (CMT) or allow beans to manage them programmatically (BMT).

Q15. What is ACID property?

Ans: ACID stands for: Atomicity (all or nothing — entire transaction succeeds or rolls back), Consistency (data moves from one valid state to another), Isolation (concurrent transactions don't interfere with each other), Durability (committed transactions survive failures).

Q16. What is container-managed transaction?

Ans: In Container-Managed Transactions (CMT), the EJB container automatically begins, commits, or rolls back transactions based on the @TransactionAttribute annotation. Transaction attributes: REQUIRED, REQUIRES_NEW, MANDATORY, NEVER, NOT_SUPPORTED, SUPPORTS.

Q17. What is bean-managed transaction?

Ans: In Bean-Managed Transactions (BMT), the bean itself controls transactions programmatically using UserTransaction API: utx.begin(), utx.commit(), utx.rollback(). Annotated with @TransactionManagement(TransactionManagementType.BEAN). Gives more control but more code.

Q18. What is JNDI?

Ans: JNDI (Java Naming and Directory Interface) is a Java API for directory services that allows Java applications to look up data and objects by name. EJBs are registered in the JNDI namespace; clients look them up: (MyBean) ctx.lookup('java:comp/env/ejb/MyBean').

Q19. What is dependency injection?

Ans: Dependency injection in EJB allows the container to automatically inject references to EJBs, resources, and other Java EE objects into a component using annotations: @EJB (inject EJB), @Resource (inject environment entry/datasource), @PersistenceContext (inject EntityManager).

Q20. What is security in EJB?

Ans: EJB security includes: authentication (verifying identity), authorization (access control), and declarative security using annotations. @RolesAllowed('admin') restricts method access to users with specific roles. @PermitAll and @DenyAll control access globally.

Q21. What is role-based security?

Ans: Role-based security (RBAC) grants access to EJB methods based on user roles (admin, user, manager). Defined declaratively using @RolesAllowed({'admin','manager'}) on EJB methods. Roles are defined in ejb-jar.xml or application deployment descriptors.

Q22. What is remote interface?

Ans: A remote interface allows EJB clients on different JVMs/machines to invoke EJB methods. In EJB 3.x, annotate a business interface with `@Remote`. Remote calls use RMI (Remote Method Invocation), involve network overhead and object serialization.

Q23. What is local interface?

Ans: A local interface allows EJB clients in the same JVM to invoke EJB methods. Annotated with `@Local`. Local calls are faster (no serialization, no network). In EJB 3.x, if no annotation is specified and a single interface exists, it defaults to local.

Q24. What is lifecycle of EJB?

Ans: Stateless bean lifecycle: Does Not Exist → create/passivate pool (setSessionContext, PostConstruct) → Method Ready Pool (execute methods) → destroy (PreDestroy). Stateful bean adds Passivated state (serialized to disk when inactive) between client calls.

Q25. What is deployment descriptor?

Ans: A deployment descriptor is an XML file that provides configuration and deployment instructions for Java EE components. For EJB: `ejb-jar.xml` defines beans, transaction attributes, security roles. Modern EJB 3.x uses annotations, making XML optional.

Q26. What is interceptor?

Ans: An interceptor is a class that intercepts business method invocations on EJBs (similar to AOP). Annotated with `@Interceptors({LoggingInterceptor.class})`. Interceptor methods use `@AroundInvoke`. Used for cross-cutting concerns: logging, auditing, performance monitoring.

Q27. What is persistence?

Ans: Persistence means storing application data permanently in a database so it survives application restarts. In Java EE, JPA (Java Persistence API) handles persistence. EJBs use `@PersistenceContext` to inject `EntityManager` for CRUD operations on `@Entity` classes.

Q28. What is ORM?

Ans: ORM (Object-Relational Mapping) automatically maps Java objects (entities) to relational database tables, eliminating manual SQL for basic CRUD. JPA is Java EE's ORM standard; Hibernate is the most popular JPA provider. Annotations: `@Entity`, `@Table`, `@Id`, `@Column`.

Q29. What is scalability in EJB?

Ans: EJB scalability is achieved through: container-managed instance pooling (stateless beans), clustering (multiple servers), load balancing, connection pooling (datasources), asynchronous processing (MDB), caching (singleton beans), and horizontal scaling across JVMs.

Q30. What are advantages of EJB?

Ans: Advantages: automatic transaction management, built-in security framework, distributed computing support, container-managed lifecycle, scalability through pooling/clustering, JMS integration, standardized Java EE specification, separation of business logic, and enterprise-grade reliability.